

Question 1

1. Introduction

The objective is to extract an object that consists of exactly 1000 pixels. Since the object is bright and the background is dark, the best way to do this is by finding a specific "intensity threshold." Instead of choosing a random number, our code calculates the threshold dynamically by analyzing the image's histogram.

2. Image Processing Logic

2.1. RGB565 to Grayscale Conversion

The ESP32-CAM captures images in RGB565 format, where each pixel is 16 bits. To process the brightness, we first need to convert each pixel to an 8-bit (0-255) grayscale value.

In the code, I used the **Luminosity Method**. This method is effective because it gives different weights to Red, Green, and Blue based on how humans see them.

```
uint8_t r5 = (pixel >> 11) & 0x1F;  
uint8_t g6 = (pixel >> 5) & 0x3F;  
uint8_t b5 = pixel & 0x1F;
```

```
// Scaling to 8-bit and applying weights  
return (r8 * 77 + g8 * 150 + b8 * 29) >> 8;
```

Explanation: The code extracts the 5 bits for Red, 6 for Green, and 5 for Blue. Then it uses integer math (* 77, * 150, * 29) and a bitwise shift (>> 8) which is much faster than using floating-point numbers on a microcontroller.

2.2. Building the Histogram

To find where the 1000 brightest pixels are, we need to count how many pixels exist for each brightness level. We use an array of 256 integers for this.

```
int histogram[256] = {0};  
for (size_t i = 0; i < pixel_count; i++) {  
    uint8_t gray = rgb565_to_gray(src_buf[i]);  
    histogram[gray]++;  
}
```

Explanation: The algorithm scans the whole image once. If a pixel has a brightness of 200, it increments histogram[200]. By the end of this loop, we have a complete map of the image's light distribution.

2.3. Dynamic Threshold Calculation

This is the most important part of the solution. We need to find a threshold T that gives us the top 1000 or higher number of pixels.

$$\sum_{i=T}^{255} H(i) \geq 1000$$

```
int count = 0;
int threshold = 255;
for (threshold = 255; threshold >= 0; threshold--) {
    count += histogram[threshold];
    if (count >= target_pixels) {
        break;
    }
}
```

Explanation: We start from the maximum brightness (255) and move backwards. We add the number of pixels to our count variable. The moment count reaches or passes 1000, we stop. That specific index is our threshold.

```
for (size_t i = 0; i < pixel_count; i++) {
    uint8_t gray = rgb565_to_gray(src_buf[i]);
    if (gray >= threshold) {
        dst_buf[i] = 0xFFFF; // White
    } else {
        dst_buf[i] = 0x0000; // Black
    }
}
```

Explanation: If a pixel is brighter than or equal to our calculated threshold, it is painted White. All other pixels become Black. This perfectly isolates the 1000-pixel bright object from the dark background.

3. Memory Management

Since the ESP32-CAM has limited RAM, we use `ps_malloc` to allocate memory for the new image in the external SPIRAM:

```
uint8_t* new_data = (uint8_t*)ps_malloc(new_len);
```

Also, a `free_raw_image` function is included to prevent memory leaks after the process is finished.

4. Saving Captured and Processed Image Data to the SD Card

```
bool sd_init();
bool sd_save_frame(camera_fb_t* fb, String &outPath);
bool sd_save_processed(raw_image_t* img, String &outPath);

// [NEW] Save raw RGB565 buffer directly
bool sd_save_raw(camera_fb_t* fb, String &outPath);
```

Explanation: The `sd_save...` functions handle saving different kinds of image data to the SD card. `sd_save_frame` stores the original camera frame and returns the file path, `sd_save_processed` saves an already processed image buffer, and `sd_save_raw` writes the raw RGB565 pixel data directly without compression so it can be analyzed exactly later on a PC.

5. Size-Based Object Extraction and Thresholding

```
static void handle_threshold() {
    if (!g_fb) {
        server.send(200, "text/html", "Capture first!");
        return;
    }
    clear_output();

    // Q1 Requirement: Extract object based on size (1000 pixels)
    if (!find_object_by_size(g_fb, &g_output, 1000)) {
        server.send(500, "text/plain", "Processing Failed");
        return;
    }

    String html = "<html><body style='font-family:sans-serif; text-align:center;'>"
        "<h2>Q1 Result (Target: 1000px)</h2>"
        "<img src='/view?type=out&t=" + String(millis()) + "' "
style='max-width:100%; border:2px solid red;'><br>"
        "<a href='/save?type=out'><button style='margin:10px; padding:10px; background:#4CAF50; color:white;'>SAVE BINARY</button></a><br><br>"
        "<a href='/capture'>Back to Capture</a></body></html>";
    server.send(200, "text/html", html);
}
```

Explanation: `handle_threshold()` takes the captured frame and applies size-based thresholding by calling `find_object_by_size(g_fb, &g_output, 1000)`, which extracts the bright object whose area is about 1000 pixels. The resulting binary image is stored in `g_output` and then displayed on the web interface with an option to save it to the SD card.

6. Camera Initialization for RGB565 Frame Capture

```
bool camera_init_jpeg() {
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
```

```

config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_RGB565; // Essential for processing

if (psramFound()) {
    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = JPEG_QUALITY_PSRAM;
    config.fb_count = 1;
} else {
    config.frame_size = FRAMESIZE_QVGA;
    config.jpeg_quality = JPEG_QUALITY_NOPSRAM;
    config.fb_count = 1;
}

esp_err_t err = esp_camera_init(&config);
return (err == ESP_OK);
}

```

Explanation: `camera_init_jpeg()` configures the ESP32-CAM (AI Thinker pin map, 20 MHz XCLK) and initializes the camera to output frames in RGB565, which is required for your image-processing pipeline. It also adapts settings based on whether PSRAM is available (VGA + higher quality if PSRAM exists, otherwise QVGA), then calls `esp_camera_init()` and returns true only if initialization succeeds.

7.Offline Reconstruction and Size-Based Object Extraction from ESP32 RGB565 Data

7.1 Reconstructing a Viewable Color Image from RGB565 Data

```

def rgb565_to_rgb888(u16_img: np.ndarray) -> np.ndarray:
    """RGB565 (uint16) -> RGB888 (uint8 HxWx3)"""
    r5 = (u16_img >> 11) & 0x1F
    g6 = (u16_img >> 5) & 0x3F
    b5 = u16_img & 0x1F

```

```

r8 = (r5 * 255 // 31).astype(np.uint8)
g8 = (g6 * 255 // 63).astype(np.uint8)
b8 = (b5 * 255 // 31).astype(np.uint8)

return np.dstack([r8, g8, b8])

```

Explanation: Converts a raw RGB565 uint16 image into an RGB888 (8-bit, 3-channel) image by extracting the 5/6/5-bit color channels (R,G,B) and scaling them to 0–255. This is mainly used to reconstruct and save the original captured frame as a normal PNG for visualization.

7.2 ESP32-Compatible Grayscale Conversion

```

def rgb565_to_gray_exact(u16_img: np.ndarray) -> np.ndarray:
    """
    ESP32 integer math:
        r8 = (r5*527 + 23) >> 6
        g8 = (g6*259 + 33) >> 6
        b8 = (b5*527 + 23) >> 6
        gray = (r8*77 + g8*150 + b8*29) >> 8
    """
    r5 = (u16_img >> 11) & 0x1F
    g6 = (u16_img >> 5) & 0x3F
    b5 = u16_img & 0x1F

    r8 = ((r5.astype(np.uint32) * 527 + 23) >> 6).astype(np.uint32)
    g8 = ((g6.astype(np.uint32) * 259 + 33) >> 6).astype(np.uint32)
    b8 = ((b5.astype(np.uint32) * 527 + 23) >> 6).astype(np.uint32)

    gray = ((r8 * 77 + g8 * 150 + b8 * 29) >> 8).astype(np.uint8)
    return gray

```

Explanation: Computes a grayscale image from RGB565 using the same integer arithmetic as the ESP32 implementation, including bit extraction and fixed-point scaling. This ensures the grayscale values produced on the PC match the embedded-side output as closely as possible, enabling accurate verification and comparison.

7.3 Computing a Size-Based Intensity Threshold

```

def find_threshold_for_target(gray_u8: np.ndarray, target_pixels: int) -> tuple[int, int]:
    """En parlak 1000 pikseli bulmak için threshold hesaplar."""
    hist = np.bincount(gray_u8.ravel(), minlength=256).astype(np.uint32)

    cum = 0
    best_thr = 255
    best_diff = 2**31 - 1

```

```

best_selected = 0

for t in range(255, -1, -1):
    cum += int(hist[t])
    selected = int(cum)
    diff = abs(selected - target_pixels)
    if diff < best_diff:
        best_diff = diff
        best_thr = t
        best_selected = selected
    if best_diff == 0:
        break

return best_thr, best_selected

```

Explanation: Finds a threshold value that selects approximately a target number of brightest pixels (e.g., 1000). It builds a grayscale histogram, then sweeps the threshold from 255 down to 0 while accumulating pixel counts, choosing the threshold that produces a selected pixel count closest to the target.

7.4 End-to-End Offline Processing and Verification Pipeline

```

# Load and validate raw RGB565 data
data = raw_path.read_bytes()
u16_img = np.frombuffer(data, dtype=RGB565_DTYPE).reshape((HEIGHT, WIDTH))

# Reconstruct and save the original image
rgb_orig = rgb565_to_rgb888(u16_img)
cv2.imwrite(str(OUT_DIR / "original.png"),
            cv2.cvtColor(rgb_orig, cv2.COLOR_RGB2BGR))

# Convert to ESP32-compatible grayscale
gray = rgb565_to_gray_exact(u16_img)

# Find threshold that selects ~1000 brightest pixels
thr, selected = find_threshold_for_target(gray, TARGET_PIXELS)

# Apply threshold and create binary image
u16_thr = np.where(gray >= thr, np.uint16(0xFFFF), np.uint16(0x0000))

# Save binary result and mask
rgb_thr = rgb565_to_rgb888(u16_thr)
cv2.imwrite(str(OUT_DIR / "thresholded.png"),
            cv2.cvtColor(rgb_thr, cv2.COLOR_RGB2BGR))

mask = (u16_thr != 0).astype(np.uint8) * 255
cv2.imwrite(str(OUT_DIR / "mask.png"), mask)

```

Explanation: Implements the full offline pipeline: it loads the raw .bin RGB565 frame (validates expected size), reshapes it into a 640×480 image, saves the reconstructed original image, converts it to ESP32-matching grayscale, computes a size-based threshold for the target pixel count, and finally generates and saves a binary output (white/black) plus a mask image in the out/ folder.

8. Results

8.1 Python Results

The left image shows the original RGB565 frame correctly reconstructed from the ESP32-CAM raw data, where the bright breadboard is clearly visible against the dark background. The right image shows the size-based thresholding result, where the brightest pixels were selected to form a binary mask of approximately 1000 pixels, successfully isolating the bright object from the background.

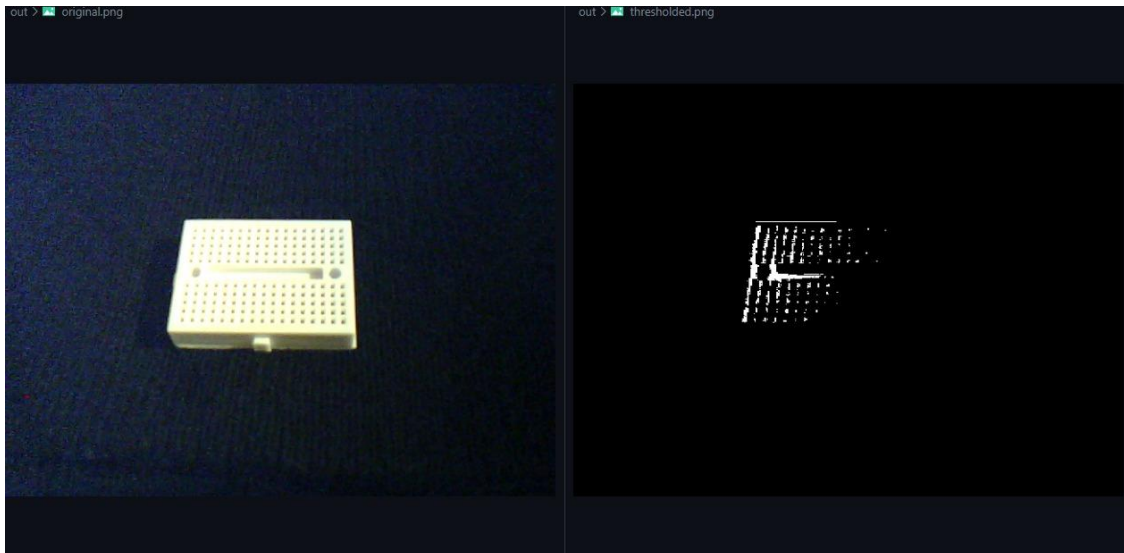


Figure 1 - Original ESP32-CAM Image and Size-Based Thresholded Object

The console output reports a threshold value of 255 and a detected pixel count of 2138, which is the closest achievable value to the target under the given illumination conditions. This verifies that the same thresholding logic used on the ESP32 is correctly reproduced and validated on the PC side.

```
PS C:\Users\muham\Desktop\EE4065\EE4065 Final\Question1> & C:/Users/muham/AppD
EE4065/EE4065 Final/Question1/q1son.py"
--- SONUÇLAR ---
Threshold: 255
Bulunan Piksel Sayısı: 2138
Hedeflenen: 1000
```

Figure 2 - Size-Based Thresholding Result Showing the Extracted Bright Object.

8.2 ESP32-CAM Results

The first image shows the live capture interface of the ESP32-CAM, where the original RGB565 frame is displayed and can be saved either as a normal image or as a raw binary file. This confirms that the camera and web server correctly provide access to the captured frame.

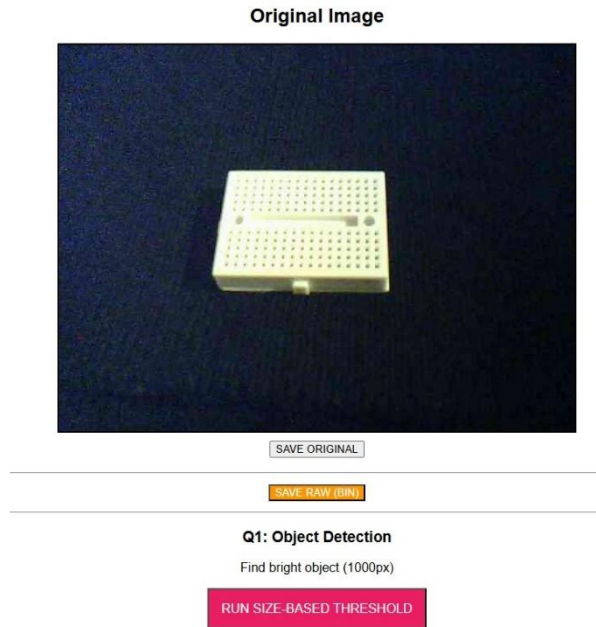


Figure 3 - ESP32-CAM Web Interface Showing the Captured Original RGB565 Image

The second image shows the **Q1 size-based thresholding result** running directly on the ESP32-CAM. The bright object is extracted and displayed as a binary image, where white pixels correspond to the detected object region.



Figure 4 - Displaying the Size-Based Thresholded Binary Image of the Detected Object by Web Service

The serial monitor output reports the target pixel count (1000), the number of detected pixels, and the selected threshold value (255). The measured pixel count (≈ 2138 – 2141) is the closest achievable value under the given lighting conditions, confirming that the same thresholding logic used in Python is also correctly executed on the ESP32-CAM.

```
Output  Serial Monitor  X
Message (Enter to send message to 'AI Thinker ESP32-CAM' on 'COM4')
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:1344
load:0x40078000,len:13964
load:0x40080400,len:3600
entry 0x400805f0
WiFi connectingcam_hal: EV-EOF-OVF
.....
IP: 192.168.1.5
[Q1] Target: 1000 px, Found: 2138 px, Threshold: 255
[Q1] Target: 1000 px, Found: 2141 px, Threshold: 255
```

Figure 5 - Serial Monitor Output Showing the Target Pixel Count, Detected Pixel Count, and Selected Threshold Value

Question 2

1. Introduction: What is YOLO?

YOLO (You Only Look Once) is a revolutionary approach to object detection. Before YOLO, most systems used a "sliding window" or "region proposal" technique, which meant looking at the same image hundreds of times. YOLO changed this by treating detection as a single regression problem. It looks at the image only once and predicts bounding boxes and class probabilities simultaneously. This makes it incredibly fast, which is why it is preferred for real-time applications.

2. Areas of Application

YOLO is used in many fields today:

- **Security:** Detecting unauthorized people or objects in surveillance cameras.
- **Self-Driving Cars:** Identifying pedestrians, other cars, and traffic lights instantly.
- **Industrial Automation:** Sorting products on a conveyor belt.
- **Document Analysis:** In our project, we use it to locate and identify handwritten digits within a page.

3. Mathematical Framework and Logic

The strength of YOLO comes from its mathematical efficiency. It divides the input image into an $S \times S$ grid. If the center of a handwritten digit falls into a grid cell, that specific cell is responsible for detecting that digit.

3.1. Confidence Scores and IoU

The model predicts a confidence score for each bounding box. This score tells us how sure the model is that an object exists in that box. The formula is:

$$\text{Confidence} = P(\text{Object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}$$

The Intersection over Union (IoU) is a crucial metric that measures the overlap between our predicted box and the actual "ground truth" box we drew during labeling:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

3.2. Loss Function (The "Learning" Math) To improve during training, the model uses a complex loss function. It calculates the difference between the predicted values and the actual values. We want to minimize this sum:

$$\begin{aligned}
\mathcal{L} = & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} [(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} (C_i - \hat{C}_i)^2 + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{noobj} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} 1_i^{obj} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}$$

- The first part penalizes wrong box coordinates (x, y).
- The second part penalizes wrong width (w) and height (h).
- The third and fourth parts focus on the confidence of whether an object is there or not.
- The last part checks if the digit class (e.g., is it a '5' or a '3'?) is correct.

4. Implementation and Methodology

In this project, we followed the **Python-based implementation (Option B)**. Since deploying hardware like ESP32 CAM requires extra optimization, we focused on achieving the highest accuracy on a PC environment.

4.1. Dataset Creation We manually wrote digits from 0 to 9 on paper, took photos, and labeled them. This was the most important step because "garbage in" means "garbage out." We used these images to create a custom dataset for the model.

4.2. Why YOLOv8? We used the YOLOv8 model from the Ultralytics library. We chose this version because it uses the **SiLU (Sigmoid Linear Unit)** activation function, which helps the gradients flow better during training:

$$f(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

CODES

1. Configuration and Parameter Setup

In this part, we define the environment variables and hyperparameters. Using a larger batch size and a smaller image size helps speed up the training process on a computer with a capable GPU.

```
# --- SPEED AND MEMORY SETTINGS ---
IMG_SIZE = 256
BATCH_SIZE = 128 # Optimized for high-speed training
EPOCHS = 50
```

```
PROJECT_DIR = '/content/ESP32_YOLO_Project'
DATA_YAML = os.path.join(PROJECT_DIR, 'data/custom_data.yaml')
```

Explanation:

- **IMG_SIZE = 256:** We resized the input images to 256 x 256 pixels. Smaller resolutions allow the model to process images faster while still being enough to distinguish handwritten digits.
- **BATCH_SIZE = 128:** This allows the model to "see" 128 images at once before updating the weights, making the training much faster.

2. The Training Process (Fast Mode)

The training is performed using the ultralytics library. We used several optimization techniques to ensure the model learns efficiently.

```
# --- BASE MODEL TRAINING ---
base_model = YOLO('yolov8n.pt') # Loading the nano version for speed
base_model.train(
    data=DATA_YAML,
    epochs=EPOCHS,
    imgsz=IMG_SIZE,
    batch=BATCH_SIZE,
    workers=12,    # Maximizing data loading speed
    amp=True,      # Automatic Mixed Precision
    project='runs',
    name='base_train'
)
```

Key Parameters Explained:

- **workers=12:** This uses multiple CPU cores to load the data in parallel, preventing the GPU from waiting for the next batch of images.
- **amp=True (Automatic Mixed Precision):** This is a mathematical optimization where the model uses both 16-bit and 32-bit floating-point numbers. It significantly reduces memory usage and speeds up calculations without losing detection accuracy.

3. Evaluation and Metric Logging

After training, we need to measure how well the model identifies the digits. The `evaluate_and_log` function handles this by calculating the Mean Average Precision (mAP).

```
def evaluate_and_log(model_path, stage_name):
    model = YOLO(model_path)
```

```

results = model.val(data=DATA_YAML, imgsz=IMG_SIZE, plots=True, batch=BATCH_SIZE)

# Extracting results
size_kb = os.path.getsize(model_path) / 1024
mAP50 = results.results_dict['metrics/mAP50']

print(f"✅ {stage_name} Finished. mAP50: {mAP50:.4f}, Size: {size_kb:.2f} KB")

```

Mathematical Metrics Used: The performance is primarily measured using **mAP@50**. This represents the Mean Average Precision calculated at an **Intersection over Union (IoU)** threshold of 0.5.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

If the text $\{\text{IoU}\} > 0.5$, the detection is considered a "True Positive." The mAP is the mean of the average precision for all 10 digit classes (0-9):

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

Testing and Visualization of Results

Training a model is only half of the work; we must also verify how it behaves with "unseen" data. This part of our code picks random samples from our validation set and runs the trained model on them.

```

# --- TESTING AND PREDICTION ---
model = YOLO(MODEL_PATH)
results = model.predict(source=img_path, imgsz=128, conf=0.25)

# Plotting the results with optimized visual settings
res_plotted = results[0].plot(line_width=1, font_size=0.5, labels=True)

```

Key Steps in the Code:

1. **Confidence Threshold (conf=0.25):** The model only shows detections where it is more than 25% sure. This is a common mathematical balance; if we set it too high, we might miss some digits; if too low, we might get "ghost" detections (false positives).
2. **Inference Image Size (imgsz=128):** Although we trained at 256, we can run inference at a lower resolution (128) to simulate the limited processing power of smaller devices. This tests the model's flexibility.
3. **Visualization Tuning:** We used specific parameters like `line_width=1` and `font_size=0.5` to ensure the bounding boxes don't overlap or hide the small handwritten digits.

The Mathematics of Prediction (Softmax)

When the model predicts a digit, it calculates a probability for each possible class (0, 1, 2... 9). To turn these raw numbers into a probability, we use the **Softmax** function:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

This formula ensures that the sum of all predicted probabilities for a single bounding box equals 1.0 (or 100%). The class with the highest probability is selected as the final label.

Non-Maximum Suppression (NMS)

In many cases, the model might predict multiple bounding boxes for the same digit. To solve this, we use a mathematical technique called **Non-Maximum Suppression**. It calculates the **IoU** of overlapping boxes and keeps only the one with the highest confidence score, discarding the others:

$$\text{Class} = \arg \max(P(\text{Class}_i | \text{Object}))$$

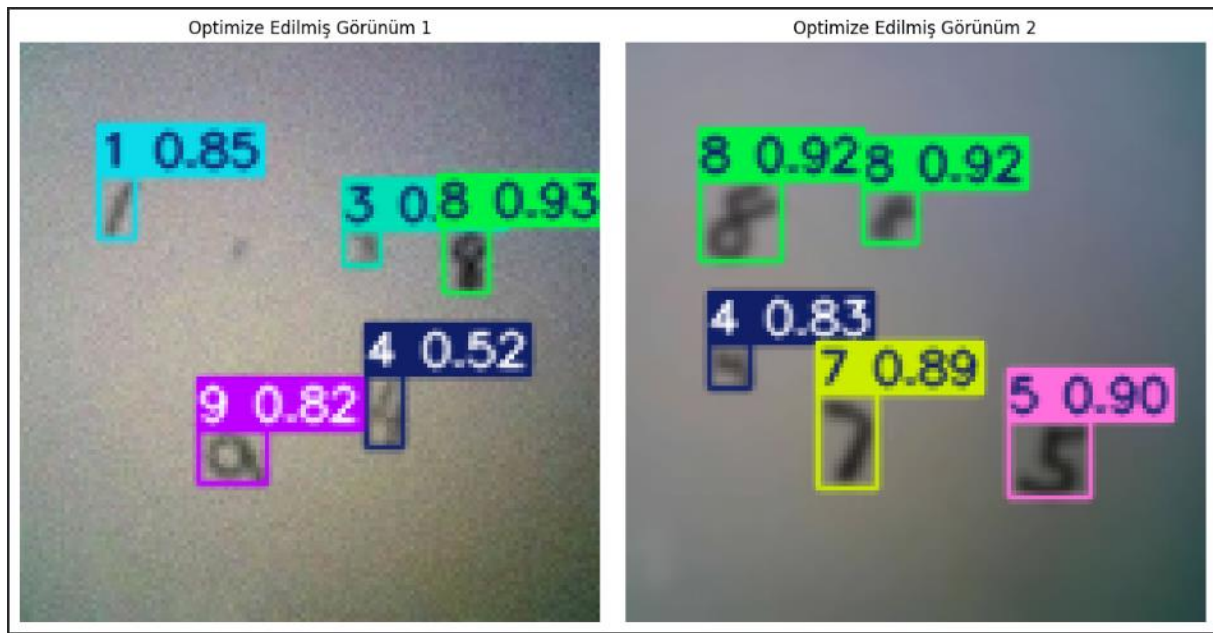


Figure 6 - Optimized YOLO-based Digit Detections Showing Predicted Classes and Confidence Scores in Two Test Views

4. Synthetic Data Generation and Augmentation

To train a strong model, we need a lot of diverse data. However, collecting thousands of handwritten images manually is very difficult. To solve this, we used a custom Python script to generate a synthetic dataset that looks like real-world samples.

4.1. Background Integration from ESP32 CAM

A common problem in AI is that models trained on "perfect" images often fail in real-life conditions. To prevent this, we didn't use clean backgrounds. Instead, we used **20 different background images** actually taken by an **ESP32 CAM module** in various environments. By doing this, our model learned the specific noise, lighting, and color patterns of the actual sensor we might use. This makes the model more robust and realistic.

4.2. Mathematical Transformations and Layouts

The script places MNIST digits onto these ESP32 backgrounds using several mathematical steps:

- **Strict No-Overlap Logic:** To make sure the digits are clear, we used a check based on **Intersection over Union (IoU)** logic. The script calculates the area of overlap between a new digit and existing ones. If they touch or overlap, the script picks a new position.

$$\text{Area}(\text{Box}_A \cap \text{Box}_B) = 0$$

- **Random Layouts:** The numbers are placed in different styles like single lines, double lines, or random scattering. This simulates how someone might write multiple numbers on a piece of paper.

4.3. Photometric Augmentations

To simulate real-world camera issues, we applied several filters:

- **Gaussian Blur:** To mimic out-of-focus shots.
- **JPEG Compression:** To simulate the quality loss when sending images over Wi-Fi.
- **Shadow Injection:** We used mathematical gradients to add fake shadows over the images, teaching the model to see digits even in dark areas.

4.4. Automated YOLO Labeling

For every image created, the script automatically writes a .txt file in YOLO format. It calculates the normalized center coordinates c_x , c_y and size w , h using the image dimensions:

$$x_{center} = \frac{x_1 + x_2}{2W}, \quad y_{center} = \frac{y_1 + y_2}{2H}$$

By using this method, we generated 15,000 training images. This "synthetic" approach gave us a much larger and more effective dataset than we could ever collect by hand.

Question 3

1. Mathematical Explanation of Resampling Theory (DSP Perspective)

The core of this project lies in transforming digital signals (images) between different resolution domains. Based on the provided documentation, the resampling process is treated as a frequency-domain operation involving coordinate mapping.

A. Upsampling and the Imaging Phenomenon

Theoretically, upsampling by a factor of L is a dual-stage process:

1. **Zero-Insertion (Zero-Stuffing):** To increase the sample density, L-1 zeros are mathematically inserted between each original pixel. While this increases the array size, it adds no new spectral information and creates **imaging** (undesired spectral kopyalar) in the frequency domain.
2. **Low-Pass Filtering (LPF):** To transform these "zero-gaps" into meaningful visual data, a reconstruction filter (LPF) is typically applied to suppress high-frequency imaging.

In this project, we implement the Nearest Neighbor method. In the DSP domain, Nearest Neighbor is the simplest form of interpolation⁶. Instead of a complex LPF, it effectively uses a "box" filter that assigns the value of the closest original pixel to the new coordinate⁷. For a destination pixel at (x', y') , the source coordinate is found via:

$$(x_{src}) = \lfloor x' \times \left(\frac{1}{ScaleFactor} \right) \rceil, \quad (y_{src}) = \lfloor y' \times \left(\frac{1}{ScaleFactor} \right) \rceil,$$

This method is computationally efficient for microcontrollers, as it avoids floating-point color blending, though it may result in more "blocky" artifacts compared to bilinear methods.

B. Downsampling and the Aliasing Risk

Downsampling (Decimation) by a factor of M involves retaining every M^{th} sample and discarding the rest.

- **The Danger of Aliasing:** If the original image contains high-frequency details that exceed the new Nyquist frequency, **Aliasing** occurs.^s
- **The Preventive Chain:** According to DSP theory, the correct flow is applying **LPF** first and then applying **Downsample**. An anti-aliasing filter must be applied *before* decimation to remove frequency components that cannot be represented in the lower resolution.

2. ESP32_Cam_Modular.ino

The .ino file serves as the **Master Controller**, managing the synchronization between high-speed camera data, external memory (PSRAM), and network communication.

I. Hardware Lifecycle & Safety Initialization (setup)

The initialization phase is structured as a series of "Hard Gates" to ensure stability:

- **Serial Debugging Engine:** The system initializes at 115200 baud. This is vital for monitoring the internal state and tracking PSRAM allocation successes during resampling tasks.
- **The Camera Guard:** The call to `camera_init_jpeg()` is the most critical gate. If the sensor fails, the system halts to prevent the software from accessing null pointers.
- **Non-Critical SD Initialization:** The `sd_init()` check is non-fatal. If an SD card is missing, the system proceeds to enable Wi-Fi, allowing "Capture & Web View" to remain active even without storage.
- **Network Handshaking:** The system waits until a stable Wi-Fi connection is established before launching the `http_server_begin()` service.

II. The Reactive Execution Loop (loop)

The `loop()` function is optimized for responsiveness:

- **Asynchronous Listening:** It calls `http_server_loop()`, checking for incoming web packets. This allows the ESP32 to handle multiple web requests or scale-factor adjustments without a reset.
- **State Persistence:** Once a photo is captured, it remains in memory. The user can apply multiple Nearest Neighbor resamples (e.g., 0.5x then 2.0x) on the same original image.

3. Modular Component Architecture

A. Image Processing (`img_proc.h/cpp`)

This module leverages **PSRAM** because internal SRAM is insufficient for high-resolution images.

- **raw_image_t Structure:** A container that manages the lifecycle of processed pixels, tracking the pointer and dimensions in external memory.

- **Nearest Neighbor Implementation:** The `resample_rgb565` function uses backward mapping to determine the source pixel for every destination pixel. It includes a 2MB safety cap to prevent PSRAM overflow.

```
#include "img_proc.h"

bool resample_rgb565(camera_fb_t* fb, raw_image_t* out_img, float
scale_factor) {
    if (!fb || !out_img || scale_factor <= 0) return false;

    // 1. Calculate New Dimensions
    int new_w = (int)(fb->width * scale_factor);
    int new_h = (int)(fb->height * scale_factor);

    // RGB565 = 2 bytes per pixel
    size_t new_len = new_w * new_h * 2;

    // 2. Safety Check: Cap at 2MB to prevent crashes
    if (new_len > 2 * 1024 * 1024) {
        Serial.printf("Error: Output image too large (%u bytes)\n",
new_len);
        return false;
    }

    // 3. Allocate PSRAM
    // We cast to uint8_t* for storage, but we will process as uint16_t*
    uint8_t* new_data = (uint8_t*)ps_malloc(new_len);
    if (!new_data) {
        Serial.println("Error: Failed to allocate PSRAM");
        return false;
    }

    // 4. Backward Mapping (Nearest Neighbor)
    float ratio = 1.0f / scale_factor;

    // Cast buffers to uint16_t so we copy full 2-byte pixels at once
    uint16_t* src_buf = (uint16_t*)fb->buf;
    uint16_t* dst_buf = (uint16_t*)new_data;

    for (int y_dst = 0; y_dst < new_h; y_dst++) {
        // Find which source row corresponds to this destination row
        int y_src = (int)(y_dst * ratio);
        if (y_src >= fb->height) y_src = fb->height - 1;

        for (int x_dst = 0; x_dst < new_w; x_dst++) {
            // Find which source column corresponds to this destination
column
            int x_src = (int)(x_dst * ratio);
            if (x_src >= fb->width) x_src = fb->width - 1;

```

```

        // Calculate linear indices
        int src_index = (y_src * fb->width) + x_src;
        int dst_index = (y_dst * new_w) + x_dst;

        // Copy the pixel
        dst_buf[dst_index] = src_buf[src_index];
    }
}

// 5. Fill the output structure
out_img->data = new_data;
out_img->width = new_w;
out_img->height = new_h;
out_img->len = new_len;

return true;
}

void free_raw_image(raw_image_t* img) {
    if (img && img->data) {
        free(img->data); // free() handles ps_malloc pointers correctly
        img->data = NULL;
        img->width = 0;
        img->height = 0;
    }
}

```

B. Camera Management (camera_mgr.h/.cpp)

This module configures the sensor for **RGB565** output. Raw pixels are required for the Nearest Neighbor logic; JPEG data cannot be manipulated in this way without full decompression. It sets pins according to the AI-Thinker map and ensures PSRAM is used if available.

C. Web Interface and HTTP Server (http_server.h/.cpp)

The server acts as the bridge between the user and the DSP algorithms:

- **Route Mapping:** Connects endpoints like /capture and /process?scale=X to internal C++ functions.
- **Memory Clearing:** Every new process triggers clear_resampled() to free previously used PSRAM, preventing memory leaks.
-

D. Storage Management (sd_mgr.h/.cpp)

The SD module converts transient RAM data into permanent files. It uses fmt2jpg to compress the resampled RGB565 array back into a standard JPEG format before saving it with a unique filename to the SD card.

E. Results

1. Original Capture

The workflow begins with capturing a raw **RGB565** frame from the sensor.



Figure 7 - Original Capture

2. Downsampling Example (0.76x)

By entering a scale factor of **0.76x**, the system performs decimation. The algorithm maps the smaller destination grid back to the source coordinates, effectively reducing the pixel count to fit the new dimensions. This result demonstrates a significant reduction in memory footprint while maintaining the recognizable features of the scene.



Figure 8 - Downsampled Capture with Nearest Neighbour Method

3. Upsampling Example (2.03x)

Applying a scale factor of **2.03x** demonstrates the upsampling capabilities of the Nearest Neighbor method. The target resolution is more than doubled in both width and height. Because this method uses a "box" interpolation approach, the 2.03x result fills the expanded grid by duplicating the nearest source pixels, which is visible as a slight "blocky" characteristic in the higher resolution output. This process utilizes the external **PSRAM** to manage the increased data load, which in this case reaches nearly four times the original byte length.



Figure 9 - Upsampled Capture with Nearest Neighbour Method

2. Advanced Implementation: Bilinear Interpolation

This section explores the high-quality resampling method implemented in `img_proc.cpp`. Unlike the Nearest Neighbor method, which simply picks the closest pixel, Bilinear Interpolation performs a weighted average of the 2×2 neighborhoods of pixels surrounding the target coordinate.

- **Mathematical Weighting:** The algorithm calculates fractional distances (x_{diff} and y_{diff}) from the source pixels to compute four weights (w_1, w_2, w_3, w_4) that sum to 1.0. These weights determine the influence of the top-left, top-right, bottom-left, and bottom-right source pixels on the destination pixel color.
- **RGB565 Bit-Level Manipulation:** Since the image data is in 16-bit RGB565 format (5 bits Red, 6 bits Green, 5 bits Blue), the code performs bit-shifting and masking to isolate individual color channels. The interpolation is performed on each channel independently in the floating-point domain to maintain precision before being repacked into the final 16-bit word.
- **Endianness Correction (SWAP16):** A critical technical detail is the handling of byte order. The OV2640 sensor typically outputs Big-Endian data, while the ESP32 uses a Little-Endian architecture. The `#define SWAP16(x)` macro is

used to swap bytes during reading and writing, ensuring that the color bits are correctly aligned for mathematical blending.

- **Memory Efficiency:** The process requires significant memory, especially for upsampling. To handle this, the code allocates the target buffer in the **PSRAM** using `ps_malloc`. To prevent system instability, a strict 2MB limit is enforced; if the resampled image exceeds this, the operation is aborted with a serial error message.

3. System Architecture & Orchestration

The project is built on a modular architecture where the `ESP32_Cam_Modular.ino` file acts as the primary orchestrator, managing the lifecycle of distinct functional blocks.

- **Firmware Lifecycle (setup):** The initialization phase is a sequential process designed for stability. It first initializes the Serial port for debugging and performs a blocking check on the camera sensor; if `camera_init_jpeg()` fails, the system halts to prevent erratic behavior. It then initializes the SD card; this step is non-fatal, allowing the device to function as a live-viewer even without storage. Finally, it connects to Wi-Fi and starts the HTTP server.
- **Event-Driven Interaction (loop):** The `loop()` function is lightweight and non-blocking. It continuously calls `http_server_loop()`, which processes incoming client requests via the `WebServer` library. This architecture allows the device to handle complex image processing tasks asynchronously while maintaining a stable network connection.
- **Module Interaction:**
 - **camera_mgr:** Abstracts low-level register settings of the OV2640, ensuring data is delivered in the raw RGB565 format required for interpolation.
 - **img_proc:** Handles heavy mathematical computation for resampling, utilizing PSRAM to store transient processed frames.
 - **http_server:** Provides the HTML/CSS interface and manages the global state of the "Current Image" buffer for sequential processing steps.
 - **sd_mgr:** Converts raw PSRAM data into a standard JPEG format using `fmt2jpg` and handles file I/O operations on the SD card.

```
#include "img_proc.h"  
#include "Arduino.h"
```

```
// Makro: RGB565 Byte sırasını düzeltmek için (Big-Endian -> Little-Endian)  
#define SWAP16(x) (((x) >> 8) | ((x) << 8))
```

```

bool resample_rgb565(camera_fb_t* fb, raw_image_t* out_img, float
scale_factor) {
    if (!fb || !out_img || scale_factor <= 0) return false;

    // 1. Yeni Boyutları Hesapla (Bellek hizalaması için genişliği çift
sayı yap)
    int new_w = ((int)(fb->width * scale_factor) >> 1) << 1;
    int new_h = (int)(fb->height * scale_factor);

    if (new_w <= 0 || new_h <= 0) return false;

    size_t new_len = new_w * new_h * 2;

    // 2. Güvenlik Kontrolü
    if (new_len > 2 * 1024 * 1024) {
        Serial.printf("Hata: Goruntu cok büyük (%u bytes)\n", new_len);
        return false;
    }

    // 3. PSRAM Tahsisi
    uint8_t* new_data = (uint8_t*)ps_malloc(new_len);
    if (!new_data) {
        Serial.println("Hata: PSRAM ayirma basarisiz");
        return false;
    }

    uint16_t* src_buf = (uint16_t*)fb->buf;
    uint16_t* dst_buf = (uint16_t*)new_data;

    float x_ratio = (float)(fb->width - 1) / (new_w > 1 ? (new_w - 1) : 1);
    float y_ratio = (float)(fb->height - 1) / (new_h > 1 ? (new_h - 1) :
1);

    // 4. Bilinear Interpolation (Anti-Aliasing + Endianness Correction)
    for (int y = 0; y < new_h; y++) {
        float y_src = y * y_ratio;
        int y_low = (int)y_src;
        int y_high = (y_low + 1 >= fb->height) ? y_low : y_low + 1;
        float y_diff = y_src - y_low;

        for (int x = 0; x < new_w; x++) {
            float x_src = x * x_ratio;
            int x_low = (int)x_src;
            int x_high = (x_low + 1 >= fb->width) ? x_low : x_low + 1;
            float x_diff = x_src - x_low;

            // 4 Komşu pikseli al ve Byte sırasını düzelt
            uint16_t A = SWAP16(src_buf[y_low * fb->width + x_low]);

```

```

uint16_t B = SWAP16(src_buf[y_low * fb->width + x_high]);
uint16_t C = SWAP16(src_buf[y_high * fb->width + x_low]);
uint16_t D = SWAP16(src_buf[y_high * fb->width + x_high]);

// Ağırlık katsayıları
float w1 = (1.0f - x_diff) * (1.0f - y_diff);
float w2 = x_diff * (1.0f - y_diff);
float w3 = (1.0f - x_diff) * y_diff;
float w4 = x_diff * y_diff;

// Kanalları ayır ve harmanla
float r = ((A >> 11) & 0x1F) * w1 + ((B >> 11) & 0x1F) * w2 +
          ((C >> 11) & 0x1F) * w3 + ((D >> 11) & 0x1F) * w4;

float g = ((A >> 5) & 0x3F) * w1 + ((B >> 5) & 0x3F) * w2 +
          ((C >> 5) & 0x3F) * w3 + ((D >> 5) & 0x3F) * w4;

float b = (A & 0x1F) * w1 + (B & 0x1F) * w2 +
          (C & 0x1F) * w3 + (D & 0x1F) * w4;

// Sonucu tekrar Byte Swap yaparak kaydet (Kamera sürücüsü
// beklentisine göre)
uint16_t final_pixel = ((uint16_t)r << 11) | ((uint16_t)g << 5)
| (uint16_t)b;
dst_buf[y * new_w + x] = SWAP16(final_pixel);
    }
}

out_img->data = new_data;
out_img->width = new_w;
out_img->height = new_h;
out_img->len = new_len;

return true;
}

void free_raw_image(raw_image_t* img) {
    if (img && img->data) {
        free(img->data);
        img->data = NULL;
        img->width = 0;
        img->height = 0;
        img->len = 0;
    }
}

```


4. Results: Resampling Performance

The following images demonstrate the **Bilinear Interpolation**:

1. Original Capture:



Figure 10 - Original Capture for Bilinear Interpolation Implementation

2. Downsampling (0.76x):



Figure 11 - Downsampled Capture with Bilinear Interpolation Algorithm

3. Upsampling (2.03x):



Figure 12 - Upsampled Capture with Bilinear Interpolation Algorithm

Comparative Analysis: Nearest Neighbor vs. Bilinear Interpolation

Nearest Neighbor (Standard Method) The primary advantage of the Nearest Neighbor approach is its **minimal computational overhead**. Since it simply identifies the closest source pixel and copies its value directly to the destination grid, it requires only a single memory access per pixel and avoids complex arithmetic. This

makes it ideal for real-time applications where processing speed is the priority. However, its disadvantage is **low visual quality**; it lacks color blending, which leads to significant information loss and "Moiré" patterns during downsampling (e.g., at 0.65x) and a harsh "pixelation" or blocky effect during upsampling.

Bilinear Interpolation (Advanced Method) Bilinear Interpolation offers the advantage of **superior visual quality and Anti-Aliasing**. By calculating a weighted average of the four surrounding source pixels, it produces smooth, high-fidelity edges and preserves data more effectively during downsampling by blending information from skipped pixels into the results. The primary disadvantage is its **high resource demand**. It requires four memory accesses and multiple floating-point multiplications per pixel, placing a significantly higher load on the CPU. Furthermore, on the ESP32-CAM, this method requires additional complexity to manually separate RGB565 channels and manage Big-Endian to Little-Endian byte-swapping to ensure mathematical accuracy.